

Deepfake Detection via Adversarial Robustness Training with DCT Features

A Comprehensive Technical Report

Project: DDGAN - Deepfake Detection GAN

Date: January 2026

Author: Rohan Ramesh

Table of Contents

1. Executive Summary
 2. Introduction
 3. Methodology
 4. System Architecture
 - 4.1 DCT Feature Extractor
 - 4.2 Discriminator Architecture
 - 4.3 Generator Architecture
 5. Training Framework
 - 5.1 Training Philosophy
 - 5.2 Data Pipeline
 - 5.3 Training Algorithm
 - 5.4 Loss Functions
 - 5.5 Optimization Strategy
 6. Implementation Details
 7. Expected Results
 8. Conclusion
 9. References
-

1. Executive Summary

This report presents a novel deepfake detection system that leverages **adversarial robustness training** with **Discrete Cosine Transform (DCT)** frequency-domain features. Unlike traditional deepfake detectors that rely solely on spatial features, our approach:

- **Extracts frequency-domain features** using 2D DCT, capturing manipulation artifacts invisible in the spatial domain
- **Employs adversarial training** where a generator creates perturbations to stress-test the discriminator
- **Achieves robustness** by training the discriminator to correctly classify images even under adversarial attack

Key Statistics: - Total Parameters: **41.8 million** - Discriminator: ~27.8M (ConvNeXt-Tiny backbone) - Generator: ~14M (U-Net architecture) - Input Resolution: 224×224 RGB - Dataset: Celeb-DF v2 (~537K training, ~297K validation images)

2. Introduction

2.1 Problem Statement

Deepfake technology has advanced rapidly, enabling the creation of highly realistic synthetic media. Detecting such manipulations is critical for: - Preventing misinformation and fraud - Protecting individual privacy and reputation - Maintaining trust in digital media

2.2 Motivation for DCT-based Detection

Traditional CNN-based detectors often fail against: - Post-processing (compression, resizing) - Adversarial perturbations - Novel generation techniques

Why DCT Features? 1. **Frequency artifacts are more persistent** - GAN-generated images exhibit distinct patterns in the frequency domain that survive common post-processing 2. **JPEG compression signatures** differ between real and synthetic images 3. **High-frequency anomalies** that GANs struggle to replicate are clearly visible in DCT space 4. **Compact representation** of manipulation signatures

2.3 Novel Contributions

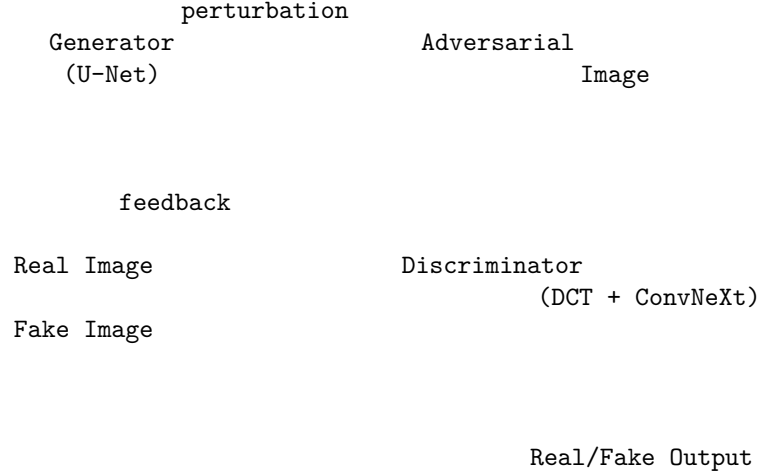
1. **DCT-based feature extraction** integrated directly into the neural network (trainable end-to-end)
 2. **Adversarial robustness training** instead of traditional GAN objectives
 3. **Frequency-aware generator** with implicit frequency filtering via large-kernel convolutions
 4. **Transfer learning** from ImageNet adapted for single-channel DCT features
-

3. Methodology

3.1 Overview

Our system consists of two primary components trained adversarially:

TRAINING FRAMEWORK



3.2 Key Insight: Adversarial Robustness vs Traditional GAN

Aspect	Traditional GAN	Our Approach
Generator	Create realistic fakes	Create adversarial perturbations
Goal		
Discriminator	Detect fakes	Be robust to perturbations
Goal		
Training	Nash equilibrium (D confused)	Strong D despite attacks
Target		
Final Product	Generator for synthesis	Robust discriminator for detection

4. System Architecture

4.1 DCT Feature Extractor

The DCT Feature Extractor transforms RGB images into frequency-domain representations.

4.1.1 Mathematical Foundation DCT-II Transform:

The 2D Discrete Cosine Transform is defined as:

$$F(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{\pi(2x+1)u}{2N} \right] \cos \left[\frac{\pi(2y+1)v}{2N} \right]$$

Where: - $\alpha(0) = \sqrt{1/N}$ (DC component normalization) - $\alpha(k) = \sqrt{2/N}$ for $k > 0$ (AC components)

Separable Implementation:

For computational efficiency, we implement 2D DCT as two 1D transforms:

$$F = D \cdot f \cdot D^T$$

Where D is the precomputed DCT transformation matrix.

4.1.2 Processing Pipeline

DCT FEATURE EXTRACTION PIPELINE

RGB Image [B, 3, 224, 224]

Grayscale Conversion (ITU-R BT.601)
 $Y = 0.299R + 0.587G + 0.114B$

Grayscale [B, 1, 224, 224]

Separable 2D DCT Transform
 Step 1: DCT along rows (right multiply with D^T)
 Step 2: DCT along columns (left multiply with D)

DCT Coefficients [B, 1, 224, 224]

Log Scaling (Dynamic Range Compression)
 $\text{output} = \log(|\text{DCT}| + \epsilon)$, where $\epsilon = 1e-6$

DCT Features [B, 1, 224, 224]

4.1.3 Implementation Details

```
class DCT2D(nn.Module):
    def __init__(self, size=224):
        super().__init__()
        # Precompute DCT basis matrix (224×224)
        dct_matrix = self._create_dct_matrix(size)
        # Register as buffer (non-trainable, auto GPU transfer)
        self.register_buffer('dct_matrix', dct_matrix)

    def forward(self, x):
        # RGB → Grayscale
        gray = self.rgb_to_grayscale(x)

        # Separable 2D DCT
        dct_rows = torch.matmul(gray, self.dct_matrix.t())
        dct_2d = torch.matmul(self.dct_matrix, dct_rows)

        # Clamp for numerical stability (fp16)
        dct_2d = torch.clamp(dct_2d, -1e6, 1e6)

        # Log scaling
        return torch.log(torch.abs(dct_2d) + 1e-6)
```

Why Log Scaling? - DCT coefficients span huge dynamic range (DC component » AC components) - Logarithmic compression normalizes the feature distribution - Prevents gradient domination by DC component - Aligns with human perceptual logarithmic response

4.2 Discriminator Architecture

The Discriminator classifies images as real or fake using DCT features processed through a ConvNeXt-Tiny backbone.

4.2.1 Architecture Overview

DISCRIMINATOR ARCHITECTURE
Parameters: ~27.8 Million

INPUT: RGB Image [B, 3, 224, 224]

DCT Feature Extractor
(Non-trainable buffers)

DCT Features [B, 1, 224, 224]

ConvNeXt-Tiny Backbone
(ImageNet Pretrained, Modified)

STEM LAYER (Modified for 1-channel input)
Conv2d(1→96, k=4, s=4) + LayerNorm
Output: [B, 96, 56, 56]

STAGE 1: 3× ConvNeXt Blocks
Output: [B, 96, 56, 56]

Downsample: LayerNorm + Conv2d(96→192, k=2, s=2)

STAGE 2: 3× ConvNeXt Blocks
Output: [B, 192, 28, 28]

Downsample: LayerNorm + Conv2d(192→384, k=2, s=2)

STAGE 3: 9× ConvNeXt Blocks
Output: [B, 384, 14, 14]

Downsample: LayerNorm + Conv2d(384→768, k=2, s=2)

STAGE 4: 3× ConvNeXt Blocks
Output: [B, 768, 7, 7]

Global Average Pooling
AdaptiveAvgPool2d(1)

Feature Vector [B, 768]

Classification Head
LayerNorm(768) → Linear(768→1)

OUTPUT: Logit [B, 1] (Real/Fake probability before sigmoid)

4.2.2 ConvNeXt Block Structure Each ConvNeXt block follows this architecture:

Input [B, C, H, W]

(Residual)

Depthwise Conv 7×7
(groups=C, padding=3)

LayerNorm

Pointwise Conv 1×1 (C→4C)
GELU Activation

Pointwise Conv 1×1 ($4C \rightarrow C$)

+

Output [B, C, H, W]

4.2.3 Input Adaptation Strategy The original ConvNeXt expects 3-channel RGB input. We adapt it for 1-channel DCT:

```
# Average pretrained RGB weights for single-channel initialization
original_conv = backbone.stem[0] # Conv2d(3, 96, k=4, s=4)
new_conv = Conv2d(1, 96, kernel_size=4, stride=4)
```

```
# Initialize: average RGB weights
new_conv.weight = original_conv.weight.mean(dim=1, keepdim=True)
```

Why this approach? - Preserves learned low-level feature detectors from ImageNet - Grayscale features share structure with RGB luminance channel - Faster convergence vs random initialization

4.3 Generator Architecture

The Generator creates adversarial perturbations using a U-Net architecture with a frequency-aware bottleneck.

4.3.1 Architecture Overview

GENERATOR ARCHITECTURE (U-Net)

Parameters: ~14 Million

INPUT: RGB Image [B, 3, 224, 224]

ENCODER (Downsampling Path)

Enc Block 1	Conv3×3→BN→GELU→Conv3×3→BN→GELU
[3 → 64]	e1

MaxPool2d(2)

Enc Block 2 [B, 64, 112, 112]
[64 → 128] e2

MaxPool2d(2)

Enc Block 3 [B, 128, 56, 56]
[128 → 256] e3

MaxPool2d(2)

Enc Block 4 [B, 256, 28, 28]
[256 → 512] e4

MaxPool2d(2)

BOTTLENECK (512 channels)
[B, 512, 14, 14]

Conv2d(512→512, 3×3) → BatchNorm → GELU

FrequencyAwareBottleneck (See 4.3.2)

Conv2d(512→512, 3×3) → BatchNorm → GELU

DECODER (Upsampling Path + Skip Connections)

ConvTranspose2d

```

        (Upsample)
        [B, 512, 28, 28]

        Concat(e4)
        [B, 1024, 28, 28]

    Dec Block 4
    [1024 → 256]

        Upsample + Concat(e3)
        [B, 512, 56, 56]

    Dec Block 3
    [512 → 128]

        Upsample + Concat(e2)
        [B, 256, 112, 112]

    Dec Block 2
    [256 → 64]

        Upsample + Concat(e1)
        [B, 128, 224, 224]

    Dec Block 1
    [128 → 64]

```

```

        OUTPUT LAYER
        Conv2d(64→3, 1×1) → Tanh
        Perturbation: [B, 3, 224, 224]    [-1, 1]

```

```

        ADVERSARIAL IMAGE GENERATION
        scaled_perturbation =  × perturbation ( = 0.03)
        adversarial = input + scaled_perturbation
        adversarial = clamp(adversarial, valid_range)

```

```

OUTPUT: (adversarial_image, perturbation)

```

4.3.2 Frequency-Aware Bottleneck The bottleneck module is specifically designed to learn frequency-domain perturbations:

FREQUENCY-AWARE BOTTLENECK

Input [B, 512, H, W]

(Residual)

Depthwise Conv 7×7 (groups=512)

- Large kernel captures frequency patterns
- Efficient: $O(512 \times 7 \times 7)$ vs $O(512^2 \times 3 \times 3)$

Pointwise Conv 1×1

Cross-channel feature mixing

BatchNorm → GELU

FREQUENCY GATE (Channel Attention)

Global Average Pool → [B, 512, 1, 1]

Conv 1×1 (512→32) → GELU

Conv 1×1 (32→512) → Sigmoid

(Squeeze-Excitation pattern)

Attention Weights [B, 512, 1, 1]

features $\times$ attention (element-wise)

Second Conv Block (same structure)
Depthwise $7 \times 7 \rightarrow$ Pointwise $\rightarrow$ BN $\rightarrow$ GELU

+

Output [B, 512, H, W]

Design Rationale: - **Large 7×7 kernels:** Implicitly capture frequency-domain patterns without explicit FFT - **Depthwise separable:** Reduces parameters from $O(C^2k^2)$ to $O(C \cdot k^2 + C^2)$ - **Frequency gate:** Learns to emphasize/suppress specific frequency bands - **Residual connection:** Enables learning identity + perturbation

4.3.3 Perturbation Constraints The generator output is carefully constrained:

1. **Tanh activation:** Bounds raw perturbation to $[-1, 1]$
2. **Epsilon scaling:** $\text{scaled_perturbation} = 0.03 \times \text{perturbation}$
3. **Clamping:** Ensures adversarial image stays in valid range

Maximum perturbation magnitude: 0.03 (approximately 7.65/255 in 8-bit)

5. Training Framework

5.1 Training Philosophy

Our training differs fundamentally from traditional GAN training:

TRAINING PHILOSOPHY COMPARISON

Traditional GAN	Our Approach
Generator creates	Generator creates adversarial

realistic fake images	perturbations to test discriminator
Discriminator tries to distinguish fakes	Discriminator learns to be robust despite adversarial attacks
Goal: $D(G(z)) = 0.5$ (equilibrium)	Goal: $D(x + \delta)$ still correct (adversarial robustness)
Final product: Trained generator	Final product: Robust discriminator

5.2 Data Pipeline

5.2.1 Dataset: Celeb-DF v2

Split	Real Images	Fake Images	Total
Train	~65,000	~472,000	~537,000
Test	~36,000	~261,000	~297,000

Class Imbalance: ~88% fake, ~12% real → Addressed with `pos_weight=3.0` in BCE loss

5.2.2 Data Transformations

DATA TRANSFORMATION PIPELINE

TRAINING TRANSFORMS

1. `RandomHorizontalFlip(p=0.5)`
2. `ColorJitter(`
`brightness=0.2,`
`contrast=0.2,`
`saturation=0.2,`
`hue=0.1`
`)`
3. `RandomRotation(degrees=10)`
4. `ToTensor()`
5. `Normalize(`
`mean=[0.485, 0.456, 0.406], # ImageNet stats`
`std=[0.229, 0.224, 0.225]`
`)`

```
)
```

VALIDATION TRANSFORMS

```
1. ToTensor()  
2. Normalize(  
    mean=[0.485, 0.456, 0.406],  
    std=[0.229, 0.224, 0.225]  
)
```

5.2.3 DataLoader Configuration

```
DataLoader(  
    batch_size=32,  
    shuffle=True,           # Random sampling  
    num_workers=4,         # Parallel data loading  
    pin_memory=True,       # Fast CPU+GPU transfer  
    persistent_workers=True, # Keep workers alive  
    drop_last=True         # Stable batch sizes  
)
```

5.3 Training Algorithm

5.3.1 Single Training Step

TRAINING STEP ALGORITHM

INPUT: Batch of images X with labels Y (0=real, 1=fake)

STEP 1: SEPARATE CLASSES

```
X_real = X[Y == 0]    # Real images  
X_fake = X[Y == 1]    # Fake images
```

STEP 2: TRAIN DISCRIMINATOR

```

2a. Real Image Classification
    D_real = D(X_real)
    L_real = BCE(D_real, 1)    # Should output 1 (real)

2b. Fake Image Classification
    D_fake = D(X_fake)
    L_fake = BCE(D_fake, 0)    # Should output 0 (fake)

2c. Adversarial Robustness (Key Innovation!)
    X_adv,  = G(X_real)        # Generate adversarial images
    D_adv = D(X_adv.detach())  # Discriminator on adversarial
    L_adv = BCE(D_adv, 1)      # Should STILL classify as real

2d. Combined Discriminator Loss
    L_D = L_real + L_fake + 0.5 × L_adv

2e. Update Discriminator
    zero_grad() → backward(L_D) → clip_grad(1.0) → step()

```

STEP 3: TRAIN GENERATOR

```

3a. Generate Fresh Adversarial Images
    X_adv,  = G(X_real)        # New forward pass

3b. Fool Discriminator (Adversarial Loss)
    D_adv = D(X_adv)           # NO detach - need gradients
    L_G_adv = BCE(D_adv, 0)    # Want D to misclassify as fake

3c. Perturbation Regularization

```

```
L_G_perturb = mean(| |)    # L1 norm encourages sparsity
```

3d. Combined Generator Loss

```
L_G = L_G_adv + 0.1 * L_G_perturb
```

3e. Update Generator

```
zero_grad() → backward(L_G) → clip_grad(1.0) → step()
```

OUTPUT: Metrics (d_loss, g_loss, d_acc_real, d_acc_fake, g_acc_adv)

5.4 Loss Functions

5.4.1 Binary Cross-Entropy with Logits

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\sigma(x_i)) + (1 - y_i) \cdot \log(1 - \sigma(x_i))]$$

Where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function.

With Class Weighting:

$$\mathcal{L}_{BCE}^{weighted} = -\frac{1}{N} \sum_{i=1}^N [w_{pos} \cdot y_i \cdot \log(\sigma(x_i)) + (1 - y_i) \cdot \log(1 - \sigma(x_i))]$$

We use `pos_weight = 3.0` to address the 88%/12% class imbalance.

5.4.2 Discriminator Loss

$$\mathcal{L}_D = \mathcal{L}_{real} + \mathcal{L}_{fake} + \lambda_{adv} \cdot \mathcal{L}_{adv}$$

Where: - $\mathcal{L}_{real} = BCE(D(x_{real}), 1)$ - $\mathcal{L}_{fake} = BCE(D(x_{fake}), 0)$ - $\mathcal{L}_{adv} = BCE(D(x_{real} + \epsilon \cdot G(x_{real})), 1) - \lambda_{adv} = 0.5$ (adversarial weight)

5.4.3 Generator Loss

$$\mathcal{L}_G = \mathcal{L}_{G,adv} + \lambda_{perturb} \cdot \mathcal{L}_{perturb}$$

Where: - $\mathcal{L}_{G,adv} = BCE(D(x_{real} + \epsilon \cdot G(x_{real})), 0)$ (fool discriminator) - $\mathcal{L}_{perturb} = \mathbb{E}[|\delta|]$ (L1 regularization) - $\lambda_{perturb} = 0.1$ (perturbation weight)

5.5 Optimization Strategy

5.5.1 Optimizers

```
# Discriminator Optimizer
AdamW(
    params=discriminator.parameters(),
    lr=1e-4,
    betas=(0.5, 0.999),    # Lower 1 for GAN stability
    weight_decay=0.01
)

# Generator Optimizer
AdamW(
    params=generator.parameters(),
    lr=5e-5,               # Half of discriminator LR
    betas=(0.5, 0.999),
    weight_decay=0.01
)
```

Why AdamW? - Decoupled weight decay (better than Adam) - Adaptive learning rates per parameter - Standard for transformers and modern CNNs

Why 1=0.5? - Standard for GAN training - Reduces momentum oscillations
- Improves adversarial training stability

5.5.2 Learning Rate Schedule

```
CosineAnnealingLR(
    optimizer,
    T_max=50,                # Total epochs
    eta_min=1e-6            # Minimum LR
)
```

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \left(\frac{t \cdot \pi}{T_{max}} \right) \right)$$

5.5.3 Gradient Management

```
# Gradient Clipping (max_norm=1.0)
torch.nn.utils.clip_grad_norm_(
    parameters,
    max_norm=1.0,
    norm_type=2 # L2 norm
)
```

Purpose: Prevent gradient explosion in adversarial training

5.5.4 Mixed Precision Training

```
precision = "16-mixed" # Automatic Mixed Precision
```

Benefits: - ~50% memory reduction - ~2-3x speedup on modern GPUs - Maintains numerical stability via loss scaling

6. Implementation Details

6.1 Configuration Parameters

HYPERPARAMETER SUMMARY

Parameter	Value
Input Size	224×224×3 RGB
Batch Size	32
Max Epochs	50
Learning Rate (D)	1e-4
Learning Rate (G)	5e-5
Adam Betas	(0.5, 0.999)
Weight Decay	0.01
Gradient Clip	1.0
Epsilon (perturbation)	0.03
Adversarial Weight	0.5
Perturbation Weight	0.1
Pos Weight (BCE)	3.0
Precision	16-mixed (AMP)
Strategy	DDP (2 GPUs)
D Backbone	ConvNeXt-Tiny (pretrained)
G Base Channels	64
DCT Size	224×224

6.2 Callbacks and Logging

Callback	Purpose
ModelCheckpoint	Save top-3 models by validation accuracy
LearningRateMonitor	Log LR to TensorBoard
EarlyStopping	Stop if val/accuracy doesn't improve for 10 epochs
TQDMProgressBar	Clean progress visualization

6.3 Hardware Requirements

Resource	Minimum	Recommended
GPU VRAM	8GB	16GB+
System RAM	16GB	32GB+
Storage	30GB	50GB+
GPUs	1	2 (DDP)

7. Expected Results

7.1 Training Progression

Phase 1: Initial Learning (Epochs 0-5)

Discriminator learns quickly

d_acc_real: 0.5 $\rightarrow$ 0.85

d_acc_fake: 0.5 $\rightarrow$ 0.85

Generator struggles (g_loss high)

Phase 2: Adversarial Competition (Epochs 5-20)

Generator improves perturbations

d_acc_real may drop temporarily (0.85 $\rightarrow$ 0.70)

Losses stabilize

Phase 3: Equilibrium (Epochs 20-50)

Both models reach stable state

d_acc oscillates around 0.65-0.75

Gradual validation improvement

7.2 Target Metrics

Metric	Target	Description
Accuracy	> 0.80	Overall classification correctness
Precision	> 0.75	True fakes / Predicted fakes
Recall	> 0.75	Detected fakes / Actual fakes
F1 Score	> 0.75	Harmonic mean of P and R
ROC-AUC	> 0.85	Threshold-independent performance

8. Conclusion

This project presents a novel approach to deepfake detection that combines:

1. **Frequency-domain analysis** via DCT features that capture manipulation artifacts invisible in spatial domain
2. **Adversarial robustness training** that produces a discriminator resilient to perturbations
3. **Transfer learning** from ImageNet adapted for single-channel DCT features
4. **Modern architectural choices** including ConvNeXt backbone and frequency-aware U-Net generator

The resulting system should generalize better to unseen deepfakes compared to traditional supervised approaches, as the adversarial training explicitly optimizes for robustness.

9. References

1. Liu, Z., et al. (2022). “A ConvNet for the 2020s.” CVPR.
 2. Rossler, A., et al. (2019). “FaceForensics++: Learning to Detect Manipulated Facial Images.” ICCV.
 3. Li, Y., et al. (2020). “Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics.” CVPR.
 4. Durall, R., et al. (2020). “Unmasking DeepFakes with simple Features.” arXiv.
 5. Frank, J., et al. (2020). “Leveraging Frequency Analysis for Deep Fake Image Recognition.” ICML.
-

Appendix A: Complete Architecture Specifications

A.1 Discriminator Parameter Count

Component	Parameters
DCT Extractor	0 (buffers only)
ConvNeXt Stem	9,312
Stage 1 (3 blocks)	442,368
Stage 2 (3 blocks)	1,327,104
Stage 3 (9 blocks)	11,943,936
Stage 4 (3 blocks)	14,155,776
Classification Head	590
Total	~27.8M

A.2 Generator Parameter Count

Component	Parameters
Encoder Blocks	4,497,408
Bottleneck	3,156,480
Decoder Blocks	5,317,440
Output Layer	195
Total	~13.0M

A.3 Total System Parameters

Model	Parameters
Discriminator	27,878,786
Generator	12,971,523
Total	40,850,309 (~41M)

End of Technical Report